Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

[CGPA-3.32] in Cycle 1

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No:2601010
Name:Doorva Suresh Balkawade

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None
```

```
    def append(self, value):
        new_node = Node(value)
```

```
        if self.head is None:
            self.head = new_node
            return
```

```
        temp = self.head
        while temp.next:
            temp = temp.next
```

```
        temp.next = new_node
```

```
    def traverse(self):
        temp = self.head
```

```
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
```

```
        print("None")
```

```
    def insert_at_position(self, value, position):
        new_node = Node(value)
```

```
        if position < 0:
            print("Invalid position")
            return
```

```
        if position == 0:
            new_node.next = self.head
            self.head = new_node
            return
```

```
        temp = self.head
```

```
        index = 0

        while temp and index < position - 1:
            temp = temp.next
            index += 1

        if temp is None:
            print("Invalid position")
            return

        new_node.next = temp.next
        temp.next = new_node
    def find_middle(self):
        if self.head is None:
            print("List is empty")
            return

        slow = self.head
        fast = self.head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        print("Middle node:", slow.data)

    def delete_node(self, value):
        temp = self.head
        prev = None

        while temp:
            if temp.data == value:
                break

            prev = temp
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if prev is None:
            self.head = temp.next
        else:
            prev.next = temp.next
    def reverse(self):
        prev = None
        curr = self.head

        while curr:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node

        self.head = prev
```

```
    def sum_consecutive(self):
        temp = self.head
```

```
        while temp and temp.next:
            total = temp.data + temp.next.data
            print(
                str(temp.data) + " + " +
                str(temp.next.data) + " = " + str(total)
            )
            temp = temp.next
```

```
list1 = LinkedList()
```

```
list1.append(10)
list1.append(20)
list1.append(30)
list1.append(40)
list1.append(50)
```

```
print("Original Linked List:")
list1.traverse()
list1.insert_at_position(25, 2)
print("After inserting 25 at position 2:")
list1.traverse()
list1.find_middle()
list1.delete_node(30)
print("After deleting 30:")
list1.traverse()
list1.reverse()
print("Reversed Linked List:")
list1.traverse()
print("Sum of consecutive nodes:")
list1.sum_consecutive()
```

O/p

```
Original Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> None
After inserting 25 at position 2:
10 -> 20 -> 25 -> 30 -> 40 -> 50 -> None
Middle node: 30
After deleting 30:
10 -> 20 -> 25 -> 40 -> 50 -> None
Reversed Linked List:
50 -> 40 -> 25 -> 20 -> 10 -> None
Sum of consecutive nodes:
50 + 40 = 90
40 + 25 = 65
25 + 20 = 45
20 + 10 = 30
```